

Chapter 5

Model Evaluation & Improvement

Cross-validation, grid search,
and choosing the right metric

Learning Objectives

- Evaluate models robustly with **cross-validation** and its variants
- Tune hyperparameters with **grid search** — without leaking the test set
- Understand why **accuracy misleads** on imbalanced data
- Read **confusion matrices**, precision/recall, ROC & AUC
- Pick a **metric** that matches the real-world goal

Beyond a Single Train/Test Split

So far: one `train_test_split`, `fit`, then `score`.

- The score depends on **which** points happened to land in the test set
- A "lucky" split → over-optimistic; an "unlucky" split → over-pessimistic
- We want a **stable** estimate of generalization, and the tools to **improve** it

This chapter adds **cross-validation**, **grid search**, and better **metrics**.

Part 1

Cross-Validation

k-Fold Cross-Validation

Split the data into **k folds**; each fold serves as the test set exactly once.

- Train k models, get **k scores**; k is usually **5 or 10**
- Every sample is in the test set once → the model must generalize to all of it

Report the **mean** (and the spread) of the k scores.

Cross-Validation in scikit-learn

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(logreg, iris.data, iris.target, cv=5)
scores          # [0.967, 0.967, 0.9, 0.933, 1.0]
scores.mean()   # 0.96
```

- `cv` sets the number of folds
- The **mean** summarizes performance; the **range** (0.90–1.00) shows sensitivity to the split

Benefits of Cross-Validation

- **More reliable** than one split — no single lucky / unlucky test set
- Reveals how **sensitive** the model is to the data (variance across folds)
- Uses data **efficiently**: 5-fold trains on 80% each time (vs 75% for one split)

✗ **Cost**: trains $\sim k$ models $\rightarrow$ roughly $k\times$ slower

⚠ *Cross-validation **evaluates** an algorithm; it does **not** produce a model to deploy.*

Stratified k-Fold

Plain k-fold can fail when data is **ordered by class**.

- `iris` is sorted by species → fold 1 would be all class 0
- Test = class 0, train = classes 1 & 2 → **0% accuracy!**

Stratified k-fold keeps each class's proportion the same in every fold.

- **Default for classification**; regression uses plain `KFold`

More Control: KFold & Shuffling

Pass a splitter object as `cv` to control splitting exactly.

```
from sklearn.model_selection import KFold
KFold(n_splits=3)                                # plain - 0.0 on sorted iris!
KFold(n_splits=3, shuffle=True, random_state=0)  # shuffle first → good scores
```

Shuffling removes the ordering problem; fix `random_state` for reproducibility.

Leave-One-Out & Shuffle-Split

Leave-One-Out

- k = number of samples (one point per fold)
- Slow, but good on **small** datasets
- iris: 150 iterations, mean 0.95

Shuffle-Split

- Sample `train_size` / `test_size`, repeat `n_splits` times
- Can **subsample** big data
- Stratified variant available

Cross-Validation with Groups

When samples come in **related groups**, the split must not break them.

- Emotion recognition: the **same person** in train *and* test → too easy
- Goal is generalizing to **new people** → keep each person in one side

```
from sklearn.model_selection import GroupKFold
cross_val_score(model, X, y, groups, cv=GroupKFold(n_splits=3))
```

Common in **medical** (per-patient) and **speech** (per-speaker) data.

Part 2

Grid Search

Tuning Parameters

Grid search = try **all combinations** of the parameters of interest.

- Kernel SVM (`SVC`) has two key knobs: `C` and `gamma`
- Try each $\in \{0.001, 0.01, 0.1, 1, 10, 100\} \rightarrow 6 \times 6 = 36$ combinations
- Keep the combination with the best score

Simple Grid Search

Just nested loops over the parameter values:

```
best_score = 0
for gamma in [0.001, 0.01, 0.1, 1, 10, 100]:
    for C in [0.001, 0.01, 0.1, 1, 10, 100]:
        svm = SVC(gamma=gamma, C=C).fit(X_train, y_train)
        score = svm.score(X_test, y_test)
        if score > best_score:
            best_score, best_parameters = score, {'C': C, 'gamma': gamma}
```

Reports 97% on iris — but this number is **not trustworthy**. Why?

⚠ Don't Tune on the Test Set

We tried 36 settings and picked the best **on the test set** → that score is **optimistic**.
Choosing parameters with the test set **leaks** it into the model.

Fix: a three-way split.

Set	Used for
training	fit models
validation	select parameters
test	final, one-time evaluation

iris SVM: validation 0.96, **test 0.92** — the honest number.

Grid Search with Cross-Validation

Best practice — replace the validation split with **cross-validation**.

```
from sklearn.model_selection import GridSearchCV
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100],
              'gamma': [0.001, 0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(SVC(), param_grid, cv=5)
grid.fit(X_train, y_train)      # 36 × 5 = 180 model fits
```

Still split off a **test set first** — CV replaces only the train/validation split.

Using GridSearchCV

`GridSearchCV` behaves like a model, and **refits** the best params on all training data.

```
grid.score(X_test, y_test)      # 0.97 – test set never used for tuning
grid.best_params_               # {'C': 100, 'gamma': 0.01}
grid.best_score_               # 0.97 – mean CV score on the TRAINING set
grid.best_estimator_           # the refit model
```

⚠ `best_score_` (CV on train) $\neq$ the **test** score from `score`.

The Grid-Search Workflow

The full, leak-free process for tuning **and** evaluating a model:

1. Split off a **test set** (untouched until the end)
2. Run **grid search with cross-validation** on the training set
3. Read off the **best parameters** (highest mean CV score)
4. **Refit** one model on the whole training set with those parameters
5. Report its score on the **test set** — the honest generalization estimate

`GridSearchCV` does steps 2–4 for you inside `fit`.

Analyzing the Results

Inspect `cv_results_` (→ pandas) and plot a **heat map** of `C` vs `gamma`.

- Reveals which parameters actually matter (SVC is very sensitive)
- **Ranges matter** — watch for misspecified grids:
 - **flat** color → parameter has no effect / wrong scale
 - **stripes** → only one parameter matters
 - optimum on an **edge** → widen the search

Searching Non-Grid Spaces

Some parameters are **conditional** — `gamma` is unused when `kernel='linear'`.

Pass a **list of dicts**; each is searched as its own grid:

```
param_grid = [  
    {'kernel': ['rbf'], 'C': [1, 10, 100], 'gamma': [0.01, 0.1, 1]},  
    {'kernel': ['linear'], 'C': [1, 10, 100]},  
]
```

Avoids wasting time on meaningless combinations.

Nested Cross-Validation

Wrap grid search in an **outer** cross-validation loop.

```
scores = cross_val_score(GridSearchCV(SVC(), param_grid, cv=5),  
                           iris.data, iris.target, cv=5)  
scores.mean()           # 0.98
```

- Outer loop splits; inner loop does the grid search
- Returns a **list of scores**, not a model → evaluates how well *the method* does
- Expensive: here $36 \times 5 \times 5 = 900$ fits. (Grid search is `n_jobs=-1` parallelizable.)

Part 3

Evaluation Metrics & Scoring

Keep the End Goal in Mind

Accuracy and R^2 are just **defaults** — often not what the application needs.

- Choose a metric tied to the **business impact** (fewer accidents, more revenue)
- The real goal is often hard to measure → use the closest feasible **surrogate**
- Decide the metric **before** comparing models or tuning

Why Accuracy Misleads

On **imbalanced** data, accuracy can look great while the model learns nothing.

- `digits` 9-vs-rest (9:1 imbalance): always predict "not 9" → **90%** accuracy
- `DummyClassifier(strategy='most_frequent')` : 0.90
- Decision tree: 0.92, random dummy: 0.80, logistic regression: **0.98**

Accuracy can't tell a real model from "always predict the majority."

Always Compare to a Baseline

Before trusting a score, check it beats a **dumb** model.

```
from sklearn.dummy import DummyClassifier
DummyClassifier(strategy='most_frequent').fit(X_train, y_train)    # 0.90
DummyClassifier(strategy='stratified').fit(X_train, y_train)      # 0.80
```

- `most_frequent` — always predicts the majority class
- `stratified` — random guesses matching class proportions

A good metric should rate these **near zero**. If your model barely beats them, it hasn't learned much.

Kinds of Errors

For binary classification with a **positive** and **negative** class:

- **False positive** (type I): healthy patient flagged sick → extra tests, a nuisance
- **False negative** (type II): sick patient missed → potentially **fatal**

The two errors rarely cost the same — cancer screening must **avoid false negatives**.

The Confusion Matrix

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, pred)      # rows = true, cols = predicted
```

	pred negative	pred positive
true negative	TN	FP
true positive	FN	TP

Diagonal = correct. $\text{accuracy} = (TP+TN) / (TP+TN+FP+FN)$.

Worked Example: 9-vs-Rest

Logistic regression confusion matrix on the `digits` 9-vs-rest task:

	pred "not nine"	pred "nine"
true "not nine"	TN = 401	FP = 2
true "nine"	FN = 8	TP = 39

- **Precision** = $39 / (39 + 2) = \mathbf{0.95}$
- **Recall** = $39 / (39 + 8) = \mathbf{0.83}$
- **f₁** = $2 \cdot (0.95 \cdot 0.83) / (0.95 + 0.83) \approx \mathbf{0.89}$

Accuracy alone (0.98) hid that we miss ~17% of actual nines.

Precision, Recall, f-score

Precision = $TP / (TP + FP)$

Limit **false positives**

(e.g. costly drug trials)

Recall = $TP / (TP + FN)$

Catch all **positives**

(e.g. cancer screening)

- Trade-off: predict all positive $\rightarrow$ recall 1 but precision low
- **f-score** = harmonic mean: $2 \cdot (\text{precision} \cdot \text{recall}) / (\text{precision} + \text{recall})$
- On 9-vs-rest, f_1 : most-frequent **0.00**, tree 0.55, logreg **0.89** — clearer than accuracy

The classification_report

One convenient table of precision, recall, f_1 , and support per class:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, pred, target_names=["not nine", "nine"]))
```

- **Support** = number of true samples in each class
- Each class gets to be the "positive" class in one row
- The averaged row weights classes by support

The Decision Threshold

Predictions come from thresholding a score: `decision_function > 0`, or `predict_proba > 0.5`.

```
y_pred = svc.decision_function(X_test) > -0.8    # lower threshold
```

- **Lower** the threshold → more positives → **higher recall, lower precision**
- Tune it to hit an **operating point** (e.g. "90% recall")
- ⚠ Set the threshold on a **validation set**, never the test set

Calibration

A **calibrated** model's probabilities mean what they say.

- If a calibrated model predicts **70%**, it's correct ~70% of the time
- `predict_proba` output is easier to threshold (fixed 0–1 scale)
- ⚠ Not all models are calibrated — a fully grown **decision tree** is always "100% sure," even when wrong
- Overfit models tend to be **overconfident**

Choose a threshold using the probabilities, but don't assume they're accurate uncertainties.

Precision-Recall Curve

Shows precision vs recall across **all thresholds** at once.

```
from sklearn.metrics import precision_recall_curve, average_precision_score
precision, recall, thresholds = precision_recall_curve(
    y_test, svc.decision_function(X_test))
```

- Closer to the **top-right** = better (high precision *and* recall)
- **Average precision** = area under it, in $[0, 1]$ — one-number summary
- Great for **comparing models** (SVM vs random forest) at every operating point

ROC Curve and AUC

Plots **false positive rate** vs **true positive rate** (recall) over all thresholds.

```
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thresholds = roc_curve(y_test, svc.decision_function(X_test))
roc_auc_score(y_test, svc.decision_function(X_test))
```

- Ideal curve hugs the **top-left**
- **AUC** = area under the ROC curve; 1.0 = perfect, **0.5 = random**

AUC for Imbalanced Data

AUC's random baseline is **0.5 regardless of class balance** — unlike accuracy.

digits 9-vs-rest, three gamma settings for an SVM:

gamma	accuracy	AUC
1.0	0.90	0.50 (random)
0.05	0.90	0.90
0.01	0.90	1.00

Same accuracy, wildly different models — **use AUC** on imbalanced problems.

Multiclass Metrics

Mostly the binary metrics, **averaged over classes**.

- **Accuracy**, **confusion matrix**, and **classification_report** all extend directly
- `digits` (10 classes), logistic regression: accuracy **0.953**
- Multiclass **f-score** needs an averaging strategy:
 - **macro** — equal weight per class (use if all classes matter equally)
 - **weighted** — weighted by support (the report's default)
 - **micro** — equal weight per **sample**

Regression Metrics

- **R^2** (the default `.score`) is usually the most intuitive — start here
- Alternatives when the application demands them:
 - **mean squared error (MSE)** — penalizes large errors heavily
 - **mean absolute error (MAE)** — treats all errors proportionally

For most tasks, R^2 is enough to compare regressors.

Using Metrics in Model Selection

Optimize grid search / CV for the metric you actually care about — via `scoring`.

```
cross_val_score(SVC(), X, y, scoring="roc_auc")
GridSearchCV(SVC(), param_grid, scoring="roc_auc")
```

Common values: `accuracy`, `roc_auc`, `average_precision`,
`f1` / `f1_macro` / `f1_micro` / `f1_weighted`, `r2`, `neg_mean_squared_error`.

(On 9-vs-rest, tuning for AUC finds a better model than tuning for accuracy.)

Chapter Summary

- Use **cross-validation** for stable estimates; pick stratified / grouped variants as needed
- Tune with **GridSearchCV**; keep an untouched **test set** for the final number
- **Accuracy misleads** on imbalanced data
- Read the **confusion matrix**; choose **precision, recall, f_1 , AUC** to fit the goal
- **PR / ROC curves** show all thresholds; **AUC** is robust to imbalance
- Drive selection with the right metric via `scoring`

Next: Chapter 6 — Algorithm Chains & Pipelines